If your classes satisfy these two points, the binder will loop through all the properties on your class and bind any that it can. In the broadest sense, the binder can bind any property that

- Is public
- Has a getter (the binder won't write set-only properties)
- Has a setter or, for complex types, a non-`null` value
- Isn't an indexer

Listing 10.10 shows two extensive options class with a host of different types of properties. All the properties on `BindableOptions` are valid to bind, and all the properties on `UnbindableOptions` are not.

Listing 10.10 An options class containing binding and nonbinding properties

```
public class BindableOptions
{
    public string String { get; set; }                                  ❶
    public int Integer { get; set; }                                    ❶
    public SubClass Object { get; set; }                                ❶
    public SubClass ReadOnly { get; } = new SubClass();                 ❶
    public Dictionary<string, SubClass> Dictionary { get; set; }        ❷
    public List<SubClass> List { get; set; }                            ❷
    public IDictionary<string, SubClass> IDictionary { get; set; }      ❷
    public IEnumerable<SubClass> IEnumerable { get; set; }              ❷
    public ICollection<SubClass> ReadOnlyCollection { get; }            ❷
        = new List<SubClass>();                                         ❷

    public class SubClass
    {
        public string Value { get; set; }
    }
}

public class UnbindableOptions
{
    internal string NotPublic { get; set; }                             ❸
    public SubClass SetOnly { set => _setOnly = value; }                ❸
    public SubClass NullReadOnly { get; } = null;                       ❸
    public SubClass NullPrivateSetter { get; private set; } = null;     ❸
    public SubClass this[int i] {                                       ❸
```